Heart Disease Prediction using ANN (TensorFlow)

Understanding Overfitting and Regularization Techniques

This notebook demonstrates:

1. What is Overfitting?
2. Building a Baseline ANN Model
3. Applying Regularization Techniques:
    - L2 Regularization
    - Dropout
    - Early Stopping
    - Batch Normalization
4. Comparing Training vs Testing Performance
5. Interpreting Results

What is Overfitting?

Overfitting occurs when a model learns the training data too well, including noise and small fluctuations.

Symptoms:

- Very high training accuracy
- Lower testing/validation accuracy
- Large gap between training and testing performance

Why it happens:

- Model too complex
- Too many parameters
- Training for too many epochs
- Small dataset

Goal: Improve generalization so the model performs well on unseen data.

Step 1: Import Required Libraries

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
```

Step 2: Load and Preprocess Dataset

We:

1. Load heart disease dataset
2. Separate features and target
3. Split into training and testing sets
4. Standardize features (important for ANN)

```
df = pd.read_csv("Heart_Disease/heart.csv")

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 3: Training Function

This function:

- Trains the model
- Stores training and testing accuracy
- Saves training history for plotting

```
results = []
histories = {}

def train_model(model, title, callbacks=None):

    history = model.fit(
        X_train, y_train,
```

```
            epochs=40,
            validation_data=(X_test, y_test),
            verbose=0,
            callbacks=callbacks
        )

        train_acc = history.history['accuracy'][-1]
        test_acc = history.history['val_accuracy'][-1]

        results.append([title, round(train_acc,4), round(test_acc,4)])
        histories[title] = history

        print(title)
        print("Training Accuracy:", round(train_acc,4))
        print("Testing Accuracy :", round(test_acc,4))
        print("-"*50)

        return model
```

What is verbose?

```
- verbose controls how much information is printed during training.
- verbose = 0,Silent mode,No output during training
- verbose = 1 (Default),Shows progress bar,Displays loss & accuracy after each e
- verbose = 2,Shows one line per epoch,No progress bar
```

What are Callbacks?

Callbacks are functions that run automatically during training.

They allow you to:

```
- Stop training early
- Save best model
- Reduce learning rate
- Customize training behavior
```

```
history.history['accuracy'][-1]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[23], line 1
----> 1 history.history['accuracy'][-1]

NameError: name 'history' is not defined
```

Step 4: Baseline ANN (Likely to Overfit)

We create a large neural network without regularization.

Why overfitting may occur?

- Large number of neurons
- No penalty on weights
- Model can memorize training data

```
model_base = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),
    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model_base.compile(optimizer='adam',
                   loss='binary_crossentropy',
                   metrics=['accuracy'])

model_base = train_model(model_base, "Baseline")
```

```
Baseline
Training Accuracy: 0.9917
Testing Accuracy : 0.8197
--------------------------------------------------
```

Step 5: L2 Regularization

What it does:

- Adds penalty term: sum of squared weights
- Prevents weights from becoming very large
- Encourages smoother learning

Effect:

- Reduces model complexity
- Improves generalization

```
model_l2 = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),

    layers.Dense(128, activation='relu',
                 kernel_regularizer=regularizers.l2(0.01)),
```

```
    layers.Dense(128, activation='relu',
                 kernel_regularizer=regularizers.l2(0.01)),

    layers.Dense(1, activation='sigmoid')
])

model_l2.compile(optimizer='adam',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])

model_l2 = train_model(model_l2, "L2 Regularization")
```

```
L2 Regularization
Training Accuracy: 0.9421
Testing Accuracy : 0.8525
--------------------------------------------------
```

Step 6: Dropout

What it does:

- Randomly disables neurons during training
- Prevents neurons from depending on each other

Effect:

- Reduces overfitting
- Improves robustness

```
model_dropout = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(1, activation='sigmoid')
])

model_dropout.compile(optimizer='adam',
                      loss='binary_crossentropy',
                      metrics=['accuracy'])

model_dropout = train_model(model_dropout, "Dropout")
```

```
Dropout
Training Accuracy: 0.8636
```

```
Testing Accuracy : 0.8689
--------------------------------------------------
```

Step 7: Early Stopping

What it does:

- Stops training when validation loss increases
- Prevents model from memorizing training data

Effect:

- Stops at optimal point
- Reduces overfitting

```
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

model_early = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),

    layers.Dense(128, activation='relu'),
    layers.Dense(128, activation='relu'),

    layers.Dense(1, activation='sigmoid')
])

model_early.compile(optimizer='adam',
                    loss='binary_crossentropy',
                    metrics=['accuracy'])

model_early = train_model(model_early, "Early Stopping", callbacks=[early_st
```

```
Early Stopping
Training Accuracy: 0.938
Testing Accuracy : 0.8525
--------------------------------------------------
```

Step 8: Batch Normalization

What it does:

- Normalizes activations within each batch
- Stabilizes and speeds up training

Effect:

- Reduces internal covariate shift
- Improves convergence

```
model_bn = keras.Sequential([
    keras.Input(shape=(X_train.shape[1],)),

    layers.Dense(128),
    layers.BatchNormalization(),
    layers.Activation('relu'),

    layers.Dense(128),
    layers.BatchNormalization(),
    layers.Activation('relu'),

    layers.Dense(1, activation='sigmoid')
])

model_bn.compile(optimizer='adam',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])

model_bn = train_model(model_bn, "Batch Normalization")
```

```
Batch Normalization
Training Accuracy: 0.9917
Testing Accuracy : 0.8852
--------------------------------------------------
```

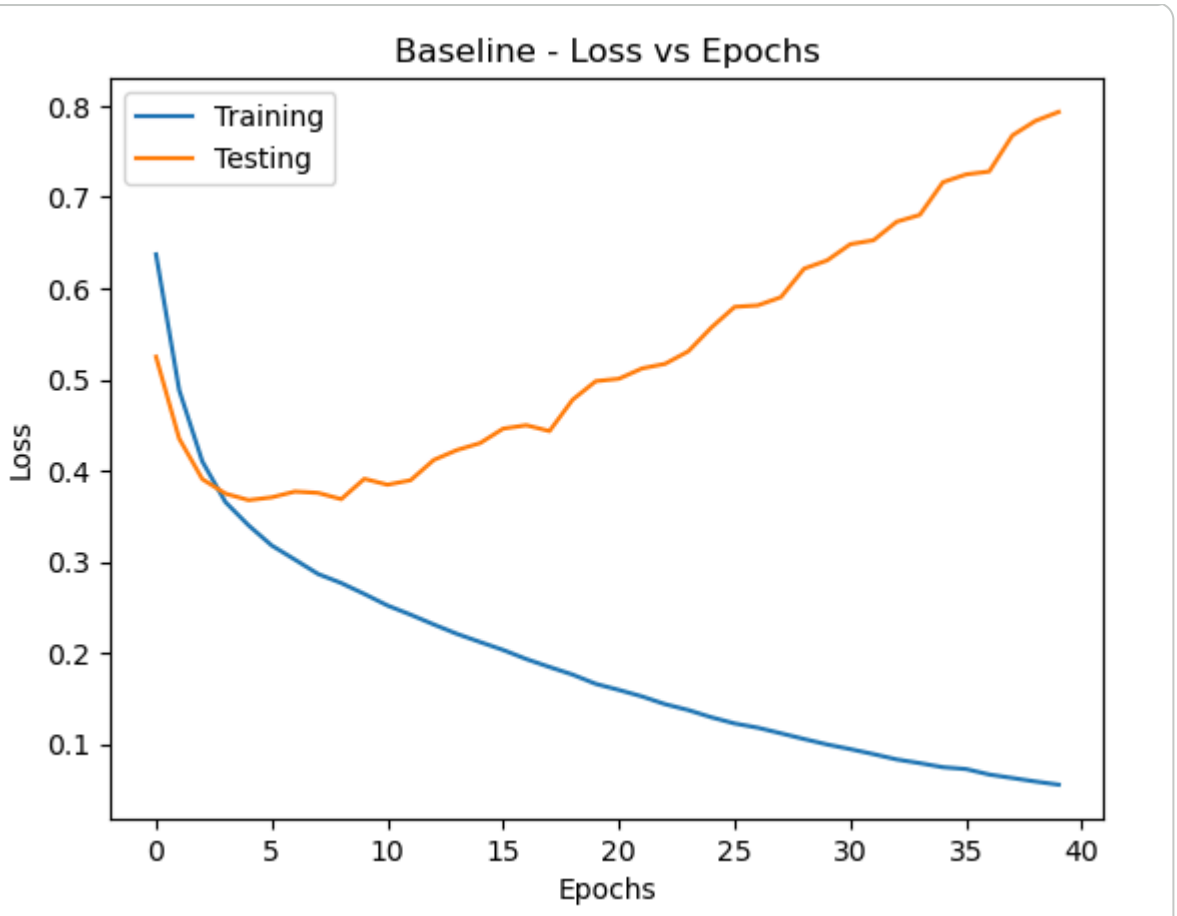
Step 9 :Model Comparision table

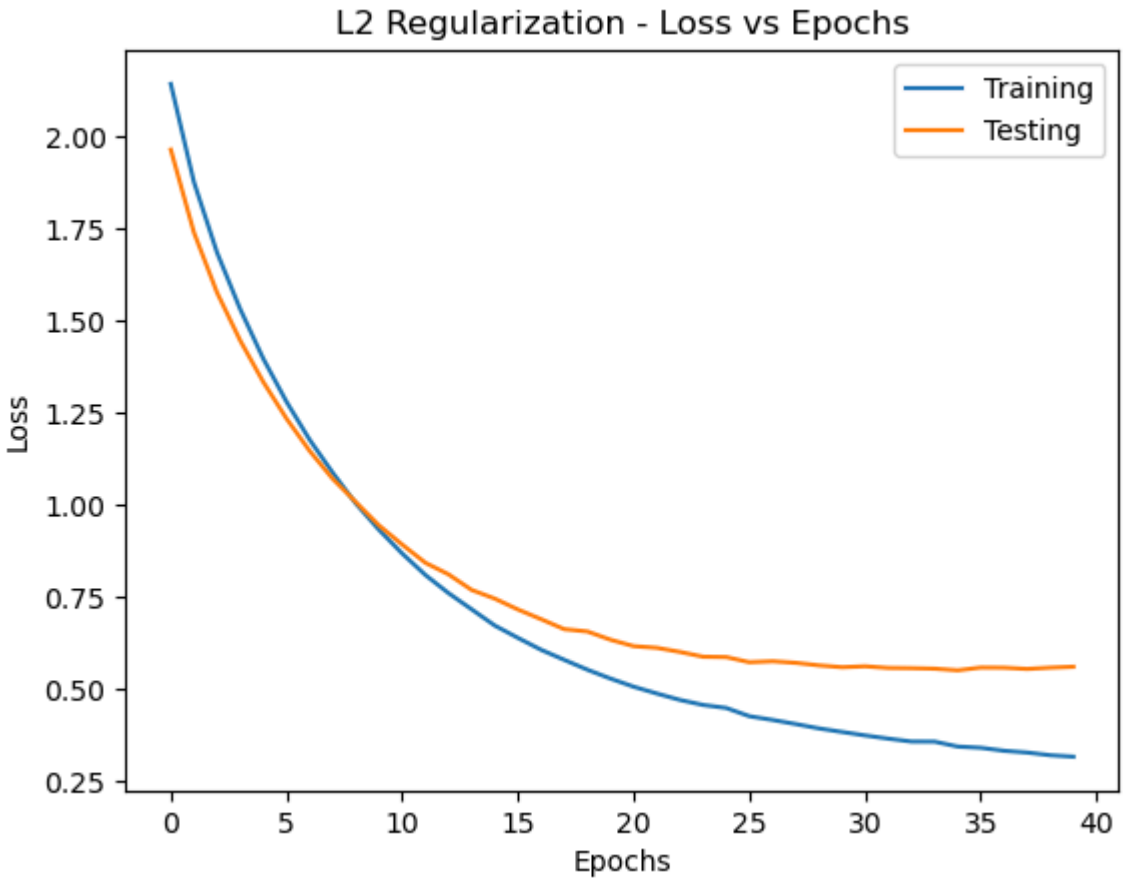
```
comparison_df = pd.DataFrame(results, columns=[
    "Model",
    "Training Accuracy",
    "Testing Accuracy"
])

comparison_df
```

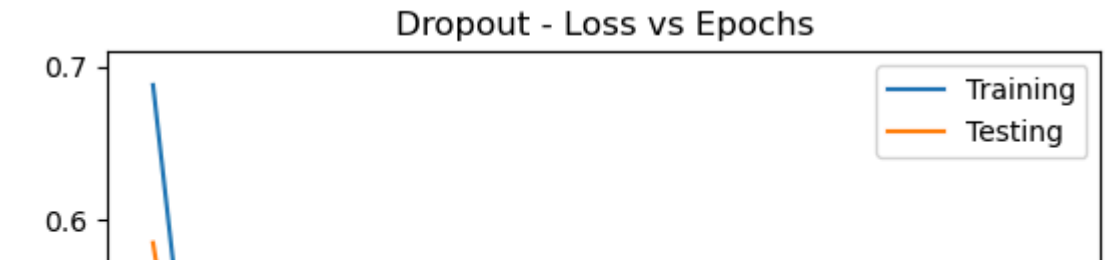
	Model	Training Accuracy	Testing Accuracy
0	Baseline	0.9917	0.8197
1	L2 Regularization	0.9421	0.8525
2	Dropout	0.8636	0.8689
3	Early Stopping	0.9380	0.8525
4	Batch Normalization	0.9917	0.8852

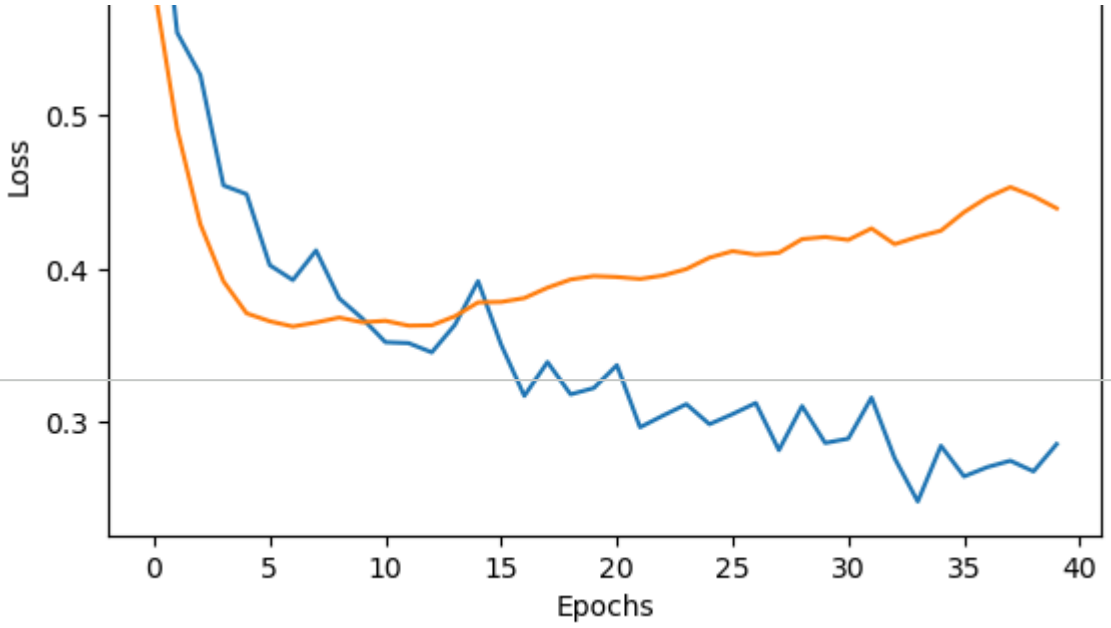
10.Loss vs Epochs (All Models)

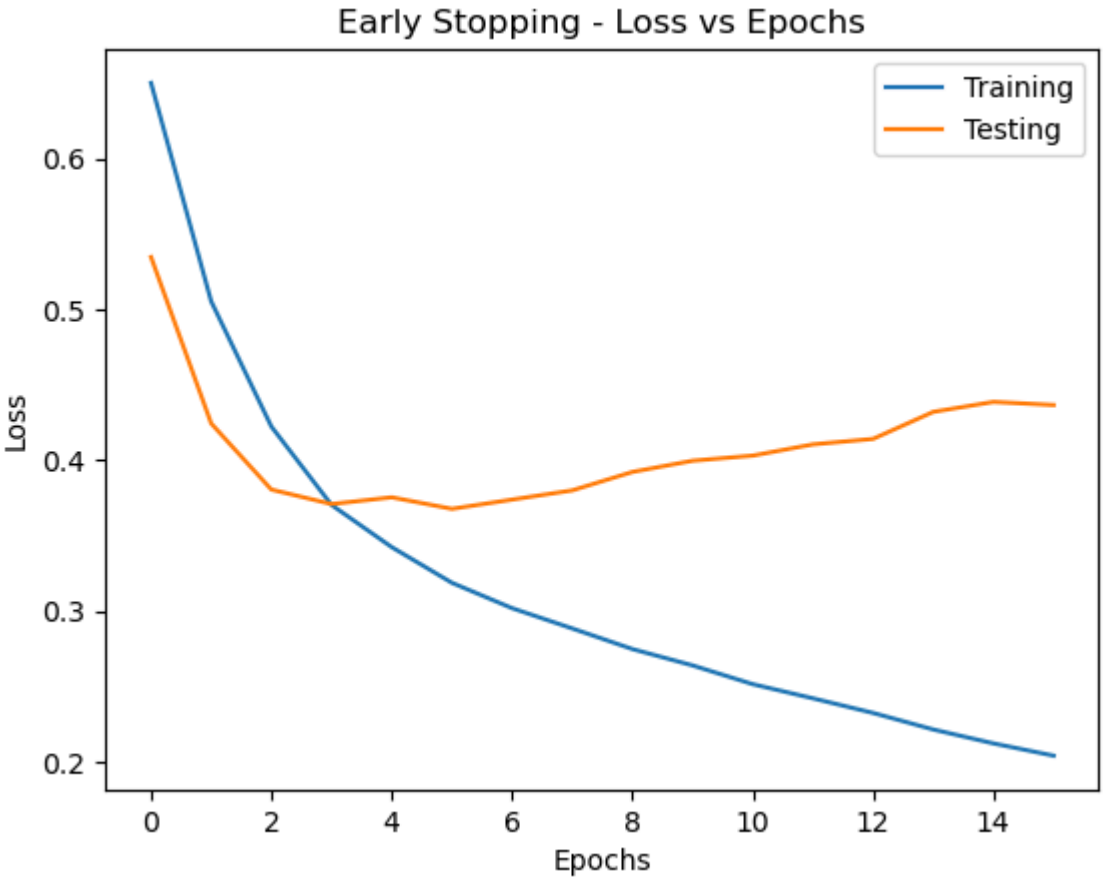
```
for title, history in histories.items():
    plt.figure()
    plt.plot(history.history['loss'])
    plt.plot(history.history['val_loss'])
    plt.title(title + " - Loss vs Epochs")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend(["Training", "Testing"])
    plt.show()
```

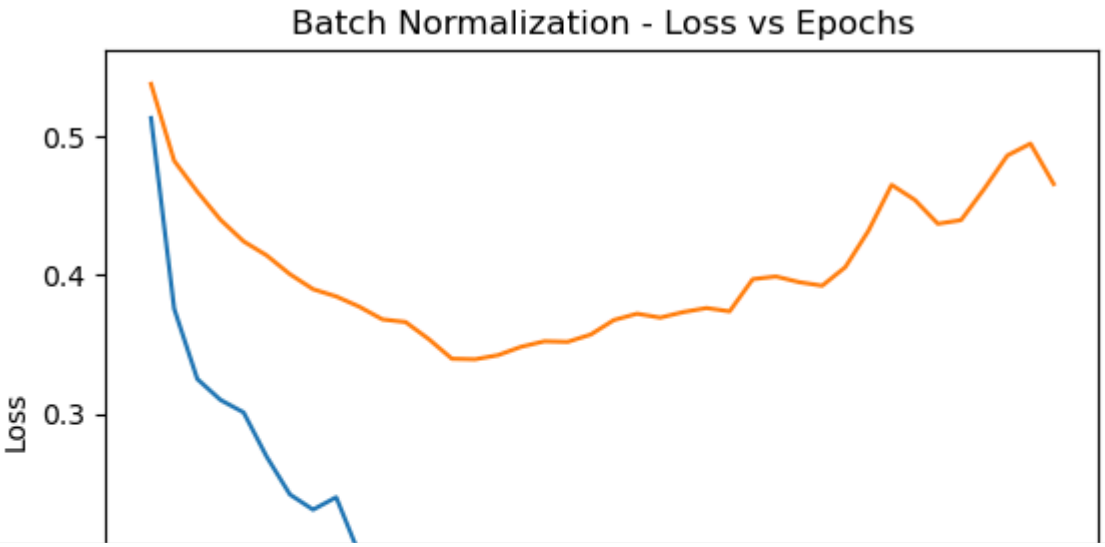

Baseline - Loss vs Epochs
Training
Testing
Loss
Epochs
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
5
10
15
20
25
30
35
40


L2 Regularization - Loss vs Epochs
Training
Testing
Loss
Epochs
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0
5
10
15
20
25
30
35
40


Dropout - Loss vs Epochs
Training
Testing
0.7
0.6


Loss
0.5
0.4
0.3
0
5
10
15
20
25
30
35
40
Epochs


Early Stopping - Loss vs Epochs
Training
Testing
Loss
0.6
0.5
0.4
0.3
0.2
0
2
4
6
8
10
12
14
Epochs


Batch Normalization - Loss vs Epochs
Loss
0.5
0.4
0.3